AI Chatbot Platform for Educational Institutions

Proof of Concept (POC) Document

CONFIDENTIAL

Document Version: 1.0

Date: May 21, 2026

Classification: Client Discussion Document

Type: Platform Discussion Document

Table of Contents

1. Executive Summary

This document presents the Proof of Concept (POC) for the **AI Chatbot Platform for Educational Institutions** — a multi-tenant SaaS platform that empowers schools, colleges, and educational bodies to deploy intelligent AI chatbots on their websites. The platform leverages Retrieval-Augmented Generation (RAG) technology to provide accurate, context-aware responses trained on institution-specific data.

> **Key Value Proposition:** A B2B2C platform where the Client (platform owner) can onboard educational institutions as subscribers, each receiving a fully isolated chatbot instance capable of answering queries about admissions, courses, fees, schedules, and more — trained entirely on their own institutional data.

Core Differentiators

- **Zero-Code Deployment:** Embeddable widget with 2-line JavaScript integration
- **Multi-Format Training:** PDF, DOCX, TXT, images (OCR), URLs, voice transcription
- **Strict Data Isolation:** Each institution's data is completely segregated
- **Lead Capture & CRM:** Automatic extraction of prospective student details
- **Human Handoff:** Seamless escalation to support agents for complex queries
- **Voice & Image Support:** Students can ask questions via voice or upload images

2. Project Understanding & Scope

2.1 Business Model (B2B2C)

Layer	Actor	Role
Platform Owner	Client (Admin)	Manages the entire SaaS platform, subscriptions, billing, and onboarding
Subscribers	Schools / Colleges	Purchase subscriptions, upload training data, customize chatbots
End Users	Students / Parents	Interact with chatbots on school websites for information

2.2 Three Core Interfaces

Super Admin Dashboard

Centralized platform control — manage all clients, subscriptions, pricing plans, revenue analytics, system configuration, and global settings.

Client Dashboard (Buyer)

Per-institution workspace — AI training, knowledge base management, chatbot customization, support tickets, lead capture, and performance insights.

Chat Widget

Lightweight embeddable JavaScript widget — glassmorphism UI, real-time responses, lead forms, voice input, image upload, typing indicators.

Voice & Image (New)

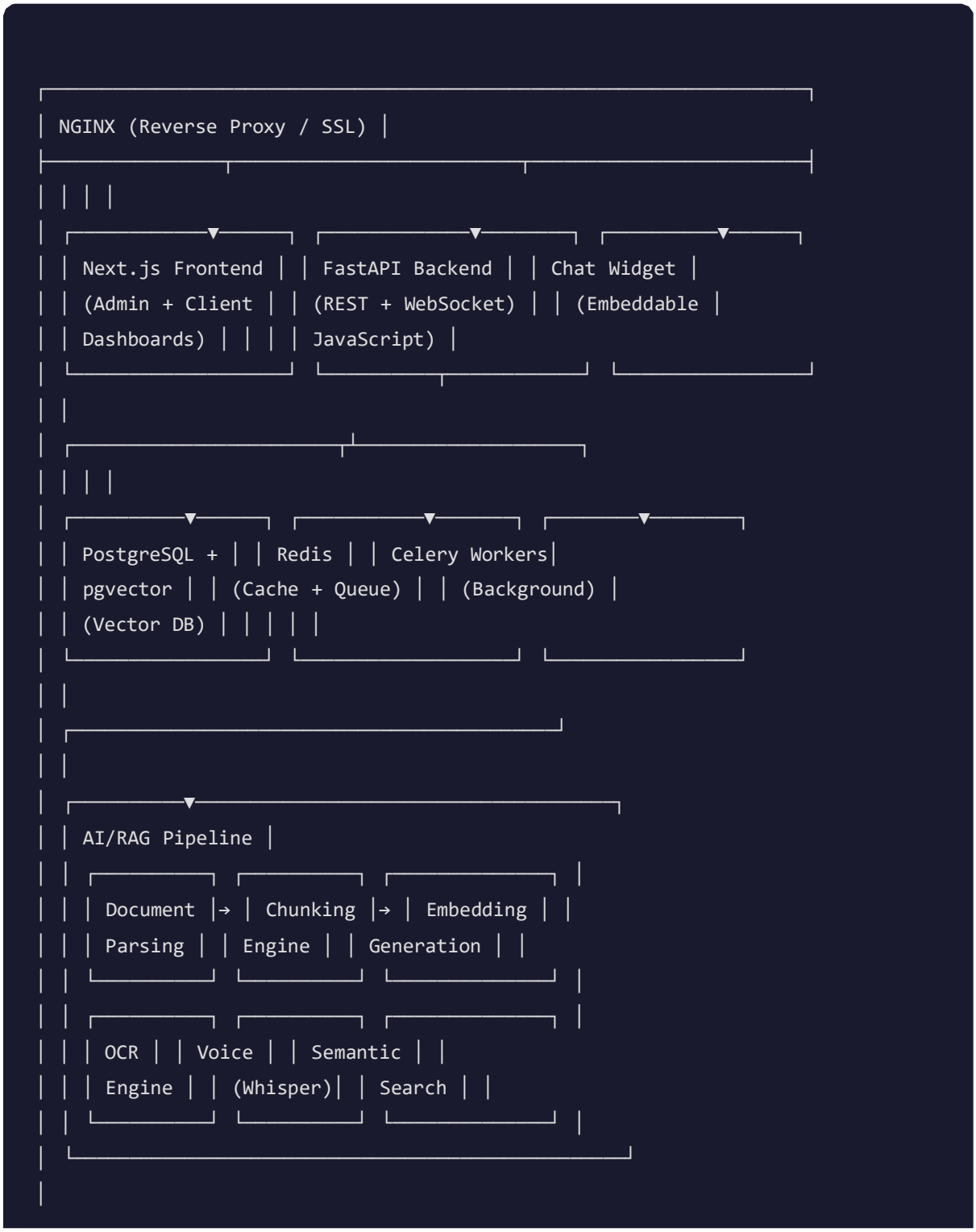
Voice-to-text input via Web Speech API/Whisper, image upload with OCR processing for extracting text from photos of documents/notices.

2.3 SRS Requirements Coverage

SRS Section	Requirement
3.1.1	Subscription & Package Management
3.1.2	Client & Workspace Isolation
3.1.3	Payment Gateway (PayU)
3.1.4	User & Role Management (RBAC)
3.1.5	Analytics & Reporting
3.1.6	System-Wide Configuration
3.2.1	AI Training Module (PDF, DOCX, OCR, URLs)
3.2.2	Knowledge Base Management
3.2.3	Category & Intent Management
3.2.4	Chatbot Behavior Customization
3.2.5	Support Ticket System
3.2.6	Automatic Lead Capture
3.2.7	Performance Insights
3.3.1	Embeddable Widget Deployment
3.3.2	Widget Customization
New	Voice Input Support
New	Image Upload & OCR in Chat

3. System Architecture

3.1 High-Level Architecture Diagram



3.2 Data Flow

> **Training Flow:** Upload → Parse (PDF/DOCX/OCR/Voice) → Chunk (sentence boundaries) → Embed (OpenAI text-embedding-3-small) → Store in pgvector
>
> **Query Flow:** User Message → Embed Query → Semantic Search (pgvector cosine similarity) → Retrieve Top-K Chunks → GPT-4o Generation with Context → Response
>
> **Voice Flow:** Audio Input → Whisper Transcription → Standard Query Flow
>
> **Image Flow:** Image Upload → Tesseract OCR → Extracted Text → Standard Query Flow

4. Module Breakdown & Feature Mapping

4.1 Super Admin Dashboard

Feature	Description	SRS Ref
Dashboard Overview	Total clients, revenue, active subscriptions, real-time metrics with charts	3.1.5
Client Management	CRUD operations, activate/deactivate, view usage, assign plans	3.1.2
Subscription Plans	Create/edit pricing tiers (Free, Premium, Enterprise), feature limits	3.1.1
User Management	Admin, Sub-admin, Client roles with granular permissions	3.1.4
Revenue & Payments	PayU integration, transaction history, invoicing, revenue charts	3.1.3
System Settings	AI model config, email templates, security policies	3.1.6
Audit Logs	Track all admin actions for compliance	5.2

4.2 Client Dashboard (School/Buyer)

Feature	Description	SRS Ref
AI Training	Drag-drop upload (PDF, DOCX, TXT, Images), URL crawler, progress tracking	3.2.1

Knowledge Base	Categorized documents, chunk viewer, re-training controls	3.2.2
Category Management	Organize content by topics — Admissions, Fees, Courses, Events	3.2.3
Chatbot Config	Tone, welcome message, FAQ suggestions, handoff rules	3.2.4
Support Tickets	Human escalation, priority levels, status tracking, internal notes	3.2.5
Lead Capture	Auto-capture from chats, export CSV, CRM integration ready	3.2.6
Analytics	Conversation trends, response times, satisfaction ratings, top queries	3.2.7
Widget Branding	Colors, logo, positioning, pre-chat forms, offline messages	3.3.2

4.3 Embeddable Chat Widget

Feature	Description
Lightweight JS Bundle	Self-contained IIFE, zero dependencies, <30KB gzipped
2-Line Integration	Config object + script tag — deployed in under 60 seconds
Glassmorphism UI	Modern frosted-glass design with smooth animations
Real-time Responses	WebSocket support for instant message delivery
Typing Indicators	Animated dots during AI processing
Lead Capture Form	Auto-triggered after N messages to collect contact info
Voice Input	Microphone button for voice-to-text queries

Image Upload	Camera/gallery button for sending images (processed via OCR)
Mobile Responsive	Full-screen mode on mobile devices
Offline Handling	Graceful fallback when connection drops

5. Technical Stack & Justification

5.1 Technology Choices

Layer	Technology	Justification
Backend API	FastAPI (Python 3.12)	Async-first, high performance, auto-docs, WebSocket support
Frontend	Next.js 15 + TypeScript	SSR/SSG, App Router, excellent DX, SEO-friendly
UI Components	Tailwind CSS + ShadCN UI	Consistent design system, accessible, customizable
Database	PostgreSQL 16 + pgvector	Production-proven, vector search built-in, no separate vector DB needed
Vector Embeddings	OpenAI text-embedding-3-small	1536-dim, excellent accuracy, cost-effective
LLM	OpenAI GPT-3.5	Best-in-class reasoning, fast response, multimodal ready
OCR	Tesseract (pytesseract)	Open-source, accurate, no per-page cost
Voice	OpenAI Whisper API	State-of-the-art transcription accuracy
Task Queue	Celery + Redis	Reliable background processing, retry logic, separate queues
Caching	Redis	Rate limiting, session cache, pub/sub for real-time
Storage	AWS S3	Scalable file storage, CDN-ready

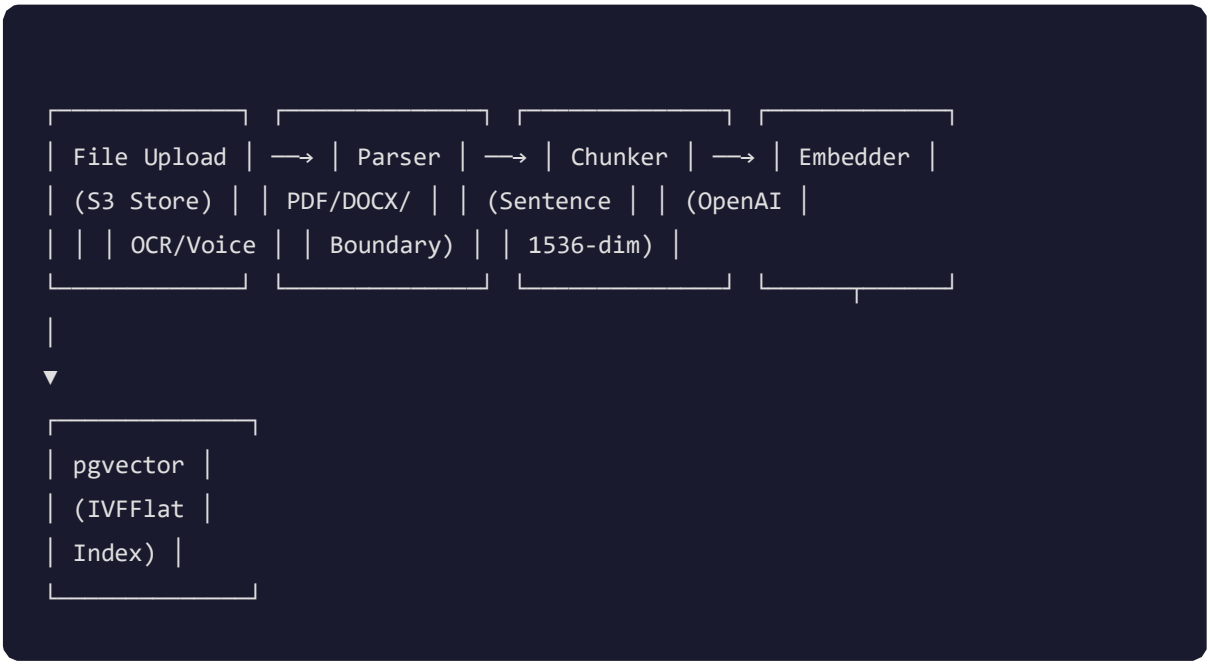
Payments	PayU	Client-specified gateway, SHA-512 verification
Auth	JWT (Access + Refresh)	Stateless, scalable, industry standard
Deployment	Docker + Docker Compose	Reproducible, scalable, cloud-agnostic
Reverse Proxy	Nginx	SSL termination, load balancing, WebSocket proxy

5.2 Why This Stack?

> **Performance:** FastAPI handles 10,000+ req/s. pgvector performs sub-50ms semantic searches.
> **Cost Efficiency:** No separate vector database subscription (Pinecone/Weaviate) — pgvector is free.
> **Scalability:** Horizontal scaling via Docker containers + Celery workers.
> **Maintainability:** Python + TypeScript are the most widely-known languages — easy to hire/onboard.

6. AI/RAG Pipeline Architecture

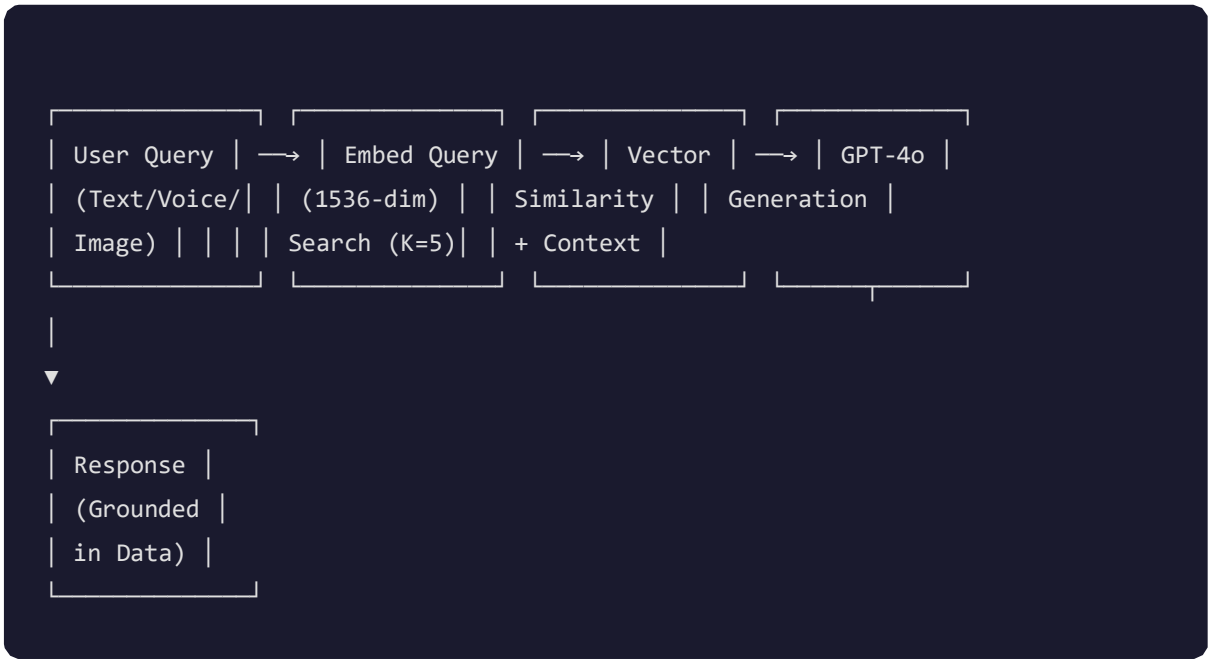
6.1 Training Pipeline (Document Ingestion)



Processing Details:

- **PDF:** PyPDF2 text extraction with fallback to OCR for scanned pages
- **DOCX:** python-docx with table and heading preservation
- **Images:** Tesseract OCR with pre-processing (deskew, threshold)
- **Voice:** Whisper API → text transcription → standard pipeline
- **URLs:** BeautifulSoup scraping → clean text extraction
- **Chunking:** 500-token chunks with 50-token overlap, sentence boundary detection
- **Embeddings:** OpenAI text-embedding-3-small (1536 dimensions)

6.2 Query Pipeline (RAG)



RAG Safeguards:

- Responses are **strictly grounded** in the institution's uploaded data
- If no relevant chunks found → polite fallback: "I don't have information about that"
- Cosine similarity threshold prevents irrelevant chunk retrieval
- Tenant isolation ensures School A's data never appears in School B's responses
- No automated retraining from user interactions (per SRS requirement)

7. Additional Capabilities: Voice & Image Upload

7.1 Voice Input

Widget Integration

Microphone button in chat widget. Uses Web Speech API for browser-native recognition or Whisper API for server-side transcription.

Use Cases

Students asking questions hands-free, accessibility for visually impaired users, mobile-first interactions.

Technical Flow:

1. User taps microphone → browser records audio
2. Audio blob sent to backend API endpoint
3. Whisper API transcribes audio to text
4. Transcribed text enters standard RAG query pipeline
5. Response displayed as text (+ optional TTS playback)

7.2 Image Upload

Widget Integration

Camera/attachment button in chat. Supports JPEG, PNG up to 10MB. Preview before send.

Use Cases

Students uploading photos of notices, fee receipts, timetables, or handwritten questions for the AI to read and respond.

Technical Flow:

1. User uploads image via widget
2. Image sent to backend → stored in S3
3. Tesseract OCR extracts text from image
4. Extracted text used as the user's query

5. RAG pipeline generates response based on extracted content

> **Example Scenario:** A parent photographs a school notice about exam schedules. The chatbot extracts the text, identifies it's about exams, and provides additional details like exam hall locations from its knowledge base.

8. Security & Multi-Tenancy Approach

8.1 Data Isolation Strategy

Layer	Mechanism
Database	Every record linked to tenant_id; all queries filtered by tenant context
Vector Store	Embeddings tagged with chatbot_id; semantic search scoped to tenant's vectors only
File Storage	S3 paths prefixed by tenant_id (e.g., /tenants/{id}/documents/)
API Layer	JWT claims include tenant_id; middleware validates access on every request
Widget	Each chatbot has unique ID; API validates chatbot ownership before responding

8.2 Authentication & Authorization

- **JWT Access Tokens:** Short-lived (30 min), contains user_id, role, tenant_id
- **Refresh Tokens:** Long-lived (7 days), secure rotation on use
- **RBAC:** Super Admin → Admin → Sub-Admin → Client Admin → Client User
- **Rate Limiting:** Per-tenant, per-endpoint throttling via Redis
- **Password Security:** bcrypt hashing with salt rounds

8.3 Compliance Measures

- All API endpoints require authentication (except widget public chat)
- Audit logging for all admin actions
- Input sanitization to prevent injection attacks

- CORS configuration restricts origins
- HTTPS enforced via Nginx SSL termination
- No PII stored in logs

9. Deployment Strategy

9.1 Container Architecture

Service	Image	Purpose
PostgreSQL	pgvector/pgvector:pg16	Primary database + vector storage
Redis	redis:7-alpine	Cache, rate limiting, Celery broker
Backend	Python 3.12-slim + Tessera t	FastAPI application server
Celery Worker	Same as backend	Background document processing
Frontend	Node 20-alpine (standalon)	Next.js SSR application
Nginx	nginx:alpine	Reverse proxy, SSL, static files

9.2 Scaling Strategy

> **Horizontal Scaling:**
> • Backend: Scale FastAPI instances behind Nginx load balancer
> • Workers: Scale Celery workers independently based on processing queue depth
> • Database: Read replicas for analytics queries; primary for writes
> • Frontend: Stateless Next.js instances, easily replicated

9.3 Recommended Cloud Infrastructure

- **Compute:** AWS EC2 / Azure VM / DigitalOcean Droplets
- **Database:** AWS RDS PostgreSQL with pgvector extension

- **Storage:** AWS S3 for documents
- **CDN:** CloudFront for widget JS delivery
- **CI/CD:** GitHub Actions → Docker Build → Deploy

10. POC Demo Scope

The following demonstrates what has been built and is ready for demonstration:

10.1 What's Ready to Demo

Module	Demo Capability
Super Admin Dashboard	Login, view metrics, manage clients, manage subscriptions
Client Dashboard	Login, upload documents, view knowledge base, manage chatbots
AI Training Pipeline	Upload PDF → auto-chunk → embed → searchable
Chat Widget	Embed on any page, ask questions, get AI responses
Lead Capture	Auto-form after 3 messages, capture name/email/phone
Ticket System	Create tickets, assign priority, track status
Voice Input	Microphone button → speech recognition → response
Image Upload	Upload image → OCR extract → AI answers
PayU Payments	Plan selection → PayU checkout → subscription activation
Multi-Tenancy	Two demo schools with isolated data

10.2 Sample Dashboard Screen

Clients

Manage school and college accounts

+ Add Client

Search clients...

Name	Contact	Status	Created	
Delhi Public School dps	admin@dps.edu.in +91 11 2634 5678	active	21 Jan 2026	
St. Xavier's College stxaviers	info@stxaviers.edu +91 22 2265 1234	active	20 Feb 2026	
Ryan International ryan	tech@ryan.edu.in +91 11 2687 9012	active	22 Mar 2026	
Amity University amity	support@amity.edu +91 120 439 2000	inactive	6 Apr 2026	
Modern School Barakhamba modern	admin@modern.edu.in +91 11 2331 5678	active	21 Apr 2026	
Kendriya Vidyalaya kv	principal@kv.edu.in +91 11 2437 8901	active	6 May 2026	

Total: 6 clients

Previous Next

Sample Screen: EduAI Super Admin Dashboard

11. Delivery Roadmap

Phase	Deliverables	Duration
Phase 1 - Core Platform	• Multi-tenant backend with RBAC • Admin + Client dashboards • AI training pipeline (PDF, DOCX, TXT) • Chat widget with basic RAG • PayU integration	4-5 Weeks
Phase 2 - Enhanced AI	• OCR for images • URL crawling • Voice input (Whisper) • Image upload in chat • Advanced analytics	2-3 Weeks
Phase 3 - Production	• Performance optimization • Security hardening • Load testing • Documentation & training • Production deployment	1-2 Weeks
Phase 4 - Enhancements	• CRM integrations • WhatsApp/Telegram channels • Advanced conversation flows • A/B testing for responses • Custom reporting	Ongoing

12. Assumptions & Dependencies

12.1 Assumptions

- Client will provide OpenAI API key (or we manage under shared account with usage billing)
- PayU merchant account will be set up by the Client
- AWS/Cloud infrastructure provisioning responsibility to be defined
- Training data (initial content) to be provided by onboarding schools
- SSL certificates will be managed (Let's Encrypt or purchased)
- No automated retraining from user chats (manual improvement only, per SRS)

12.2 Dependencies

Dependency	Provider	Impact if Delayed
OpenAI API Access	OpenAI	Core AI functionality blocked
PayU Credentials	Client	Payment flow cannot be tested
AWS S3 Bucket	Client/Us	File upload functionality blocked
SMTP Credentials	Client	Email notifications disabled
Domain & SSL	Client	Production deployment delayed

12.3 Out of Scope (for current engagement)

- Native mobile apps (iOS/Android)
- WhatsApp / Telegram bot integration
- Multi-language support (can be added in Phase 4)

- Custom LLM fine-tuning
- Data migration from existing systems